Learning Interpretable Differentiable Logic Networks

Chang Yue and Niraj K. Jha, *Fellow, IEEE*

***Abstract*—The ubiquity of neural networks (NNs) in real-world applications, from healthcare to natural language processing, underscores their immense utility in capturing complex relationships within high-dimensional data. However, NNs come with notable disadvantages, such as their "black-box" nature, which hampers interpretability, as well as their tendency to overfit the training data. We introduce a novel method for learning interpretable differentiable logic networks (DLNs) that are architectures that employ multiple layers of binary logic operators. We train these networks by softening and differentiating their discrete components, e.g., through binarization of inputs, binary logic operations, and connections between neurons. This approach enables the use of gradient-based learning methods. Experimental results on twenty classification tasks indicate that differentiable logic networks can achieve accuracies comparable to or exceeding that of traditional NNs. Equally importantly, these networks offer the advantage of interpretability. Moreover, their relatively simple structure results in the number of logic gate-level operations during inference being up to ten thousand times smaller than NNs, making them suitable for deployment on edge devices.**

***Index Terms*—Edge computing, interpretability, logic rules, machine learning, neural networks.**

I. Introduction

NEURAL networks (NNs) have become a cornerstone of modern machine learning, driving advancements in a multitude of fields. Their impressive performance has catalyzed transformative changes across various sectors, including healthcare and finance. Despite their prowess, a key drawback lies in their incomprehensible decision-making process, often termed the "black-box" problem. This lack of interpretability becomes particularly salient in applications such as medical diagnostics, where understanding the reasoning behind the NN decisions is very important. We take inspiration from work on Deep Differentiable Logic Gate Networks (DDLGNs) [1], in which the authors train classification networks consisting of logic operators by relaxing these operations to become differentiable. They randomly initialize a network such that each logic neuron is connected only to two inputs from its previous layer and then employ real-valued logic and relaxed logic operations to find the role each logic neuron should play. They can extract logic rules from the resulting networks, thus making the decision process interpretable. In some classification tasks, their model achieves comparable inference accuracies to regular Multi-Layer Perceptrons (MLPs) while operating at speeds orders of magnitude faster. In a DDLGN, the logic gate search part is differentiable; however, the network needs a fixed structure initialized at the beginning and only accepts binary inputs. We develop methods that overcome all these limitations by making all parts of the network differentiable. Moreover, we observe further performance improvement and model size reduction.

We propose a method that can learn networks consisting of logic operators, trainable through gradient-based optimization. We present a simplified example of such a network in Fig. 1 and refer to it as a differentiable logic network (DLN) because it makes decisions based on logic rules. We give examples of post-processed DLNs later. A DLN mainly consists of neurons that perform binary logic operations. Given a sample input, it first binarizes continuous features, then passes the binarized sample (which includes one-hot encoded versions of categorical features) through layers of logic operators, and finally counts the logic rules that have been triggered to determine which class this sample belongs to. Our training method can find both the network structure, i.e., connections between layers, and the Boolean function that each neuron should perform. In summary, we can train interpretable logic rule-based networks from scratch.

Another advantage that DLNs have is computational efficiency. The models that perform well generally have a small number of neurons. Moreover, connections in DLNs are sparse. In contrast, in traditional MLPs, neurons between layers are fully connected, resulting in links that are quadratic in hidden layer sizes. In addition, operations in our DLNs are also simple: for all neurons, output values are one-bit; ThresholdLayers merely perform comparisons, which can be implemented by a fast comparator; LogicLayer neurons only perform basic logic operations; and SumLayer neurons only perform binary aggregation, which can also be efficiently implemented.

In this article, we propose a novel method for training DLNs. The resulting network is accurate, interpretable, and efficient. The main contributions are as follows:

- We develop methodologies to make all components of DLNs differentiable, making them trainable from scratch.

Received 23 June 2024; revised 5 August 2024; accepted 8 September 2024. Date of publication 17 September 2024; date of current version 8 November 2024. This work was supported by NSF under Grant CNS-1907831. The review of this article was arranged by Associate Editor Yang Yi. *(Corresponding author: Chang Yue.)*

The authors are with the Department of Electrical and Computer Engineering, Princeton University, Princeton, NJ 08544 USA (e-mail: cyue@princeton.edu; jha@princeton.edu).

Digital Object Identifier 10.1109/TCASAI.2024.3462303

TABLE I
LIST OF ALL REAL-VALUED BINARY LOGIC OPERATIONS. ADAPTED FROM DDLGN [1]

ID	Operator	Real-valued	00	01	10	11
0	False	0	0	0	0	0
1	$A \wedge B$	$A \cdot B$	0	0	0	1
2	$\neg(A \Rightarrow B)$	$A - AB$	0	0	1	0
3	A	A	0	0	1	1
4	$\neg(A \Leftarrow B)$	$B - AB$	0	1	0	0
5	B	B	0	1	0	1
6	$A \oplus B$	$A + B - 2AB$	0	1	1	0
7	$A \vee B$	$A + B - AB$	0	1	1	1
8	$\neg(A \vee B)$	$1 - (A + B - AB)$	1	0	0	0
9	$\neg(A \oplus B)$	$1 - (A + B - 2AB)$	1	0	0	1
10	$\neg B$	$1 - B$	1	0	1	0
11	$A \Leftarrow B$	$1 - B + AB$	1	0	1	1
12	$\neg A$	$1 - A$	1	1	0	0
13	$A \Rightarrow B$	$1 - A + AB$	1	1	0	1
14	$\neg(A \wedge B)$	$1 - AB$	1	1	1	0
15	True	1	1	1	1	1

- We present methodologies to train interpretable DLNs efficiently and to simplify them after training.
- We conduct experiments and compare our model with various machine learning models in terms of inference accuracy and computation cost.

The article is structured as follows. Section II discusses related work. Section III presents the methodology. Section IV showcases experimental results. Section V provides a conclusion and thoughts about future research directions.

II. RELATED WORK

In this section, we introduce techniques related to our methodologies and prior works on logics, differentiability, interpretable machine learning, and efficient machine learning.

Logic Representation: Boolean logic rules perform discrete operations like AND and XOR on binary inputs to produce binary outputs. They can be described by truth tables. In our designs, we limit the number of inputs to two. To remove reliance on just the $0/1$ values that are inherent to Boolean logic, fuzzy logic [2] can be employed to enable the truth values of variables to be any real number between 0 and 1. This interpretation [3], [4] of logic operations extends traditional binary logic into the probabilistic realm, enabling differentiability. For example, $P(A \wedge B)$ can be realized as $P(A) \times P(B)$. We employ the same set of real-valued logic operations that DDLGN [1] uses; thus, we have directly adapted Table I from DDLGN. It lists all the $2^{2^2} = 16$ Boolean functions that are possible with two inputs.

Differentiability: This is a cornerstone of gradient-based optimizations. As discussed earlier, real-valued logics [2] extend binary logic to enable continuity. Softmax is a differentiable approximation to argmax; it converts logits into probabilities. Gumbel-Softmax [5] adds Gumbel noise to the logits before applying the Softmax function to enable differentiable sampling. We employ the Softmax function to determine incoming links and types of logic operators. The Straight-Through Estimator (STE) [6] passes the values of non-differentiable functions while backpropagating their approximated differentiable gradients. It provides an alternative way to estimate gradients; we have found it to be helpful in training DLNs.

Interpretable Machine Learning: Numerous attempts have been made to explain the working mechanisms of black-box models, such as LIME [7], saliency maps [8], Shapley additive explanations [9], and class activation maps [10]. However, Rudin [11] demonstrates that these post-hoc approaches are often unreliable and can be misleading. A growing body of work is focused on developing inherently interpretable models, such as sparse decision trees [12] and scoring systems [13]. These methods, in general, face computational challenges because of their discrete structure. Neuro-symbolic methods bridge symbolic and connectionist approaches. Richardson et al. [14] propose Markov logic networks (MLNs) that combine first-order logic with probabilistic graphical models. MLNs employ the Markov chain Monte Carlo method for inference. However, this approach is not scalable to large networks. Riegel et al. [15] introduce Logical Neural Networks (LNNs) that build a differentiable symbolic logic network first, and then learn the weights for each node. LNNs require hand-crafted symbolic graphs, which may necessitate extensive domain knowledge. Research efforts are also geared at learning logic rules using differentiable networks. Wang et al. [16] present the Rule-based Representation Learner by projecting rules into a continuous space and propose Gradient Grafting to differentiate the network. Our methodology, on the other hand, makes all parts of the NN differentiable and can be optimized by vanilla gradient descent. Payani et al. [17] introduce Neural Logic Networks (NLNs) to learn Boolean functions and develop several types of layers that perform logic operations. Our methodology features flexible neurons and connections. Shi et al. [18] also proposed a method named NLN that learns logic variables as vector representations and logic operations as neural modules, regularized by logical rules. Our approach learns the network structure and logic functions through relaxed differentiation, without any predefined modules. Another work related to our method is the Gumbel-Max Equation Learner [19]. The authors apply continuous relaxation to the network structure via Gumbel-Max; our methodology differs in terms of search algorithms. In the Wide & Deep Model [20] work, the authors find that sometimes it is better to bring the input closer to the output layer; this can also make the decision process more interpretable. We have found this strategy to be helpful. In summary, our method emphasizes flexibility and simplicity of use.

Efficient Machine Learning: Techniques like pruning [21], [22], quantization [23], [24], [25], and knowledge distillation [26], [27] aim to create smaller models that retain the performance characteristics of larger ones, although these approaches do not necessarily enhance model interpretability. Application-Specific Integrated Circuits and Field-Programmable Gate Arrays (FPGAs) sometimes offer highly specialized and task-specific optimizations. Umuroglu et al. [28] demonstrate that task-specific FPGAs can accelerate inference for binarized NNs. Our logic operation-based models can similarly achieve high efficiency on specialized hardware.

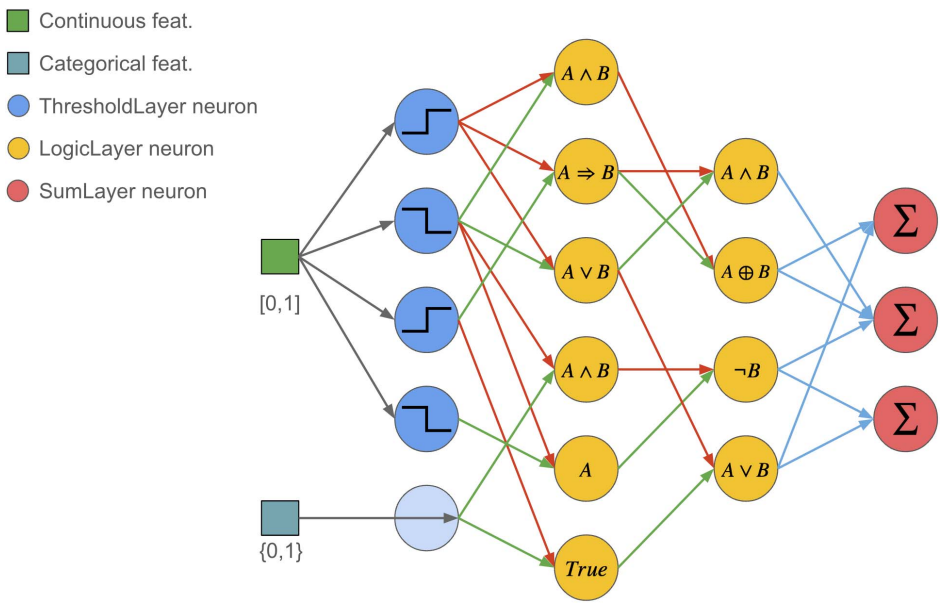


Fig. 1. A simplified DLN example. It takes input samples and binarizes continuous variables through a ThresholdLayer. It then passes the binary vector to layers of two-input Boolean logic operators. Finally, it counts logic rule triggers to determine the sample's class.

III. Methodology

In this section, we describe our method in detail, beginning with an overview, followed by details of each training step.

Fig. 1 illustrates a sample DLN. It consists of three types of layers: a ThresholdLayer at the input to binarize continuous input features into binary $0/1$ values using threshold functions, multiple LogicLayers consisting of two-input logic operators for performing logic operations, and a SumLayer at the end to aggregate the output from the last LogicLayer to determine the class. We can directly extract logic rules from the network, thereby making its predictions interpretable. Our DLN is similar to conventional MLPs in terms of input and output: both accept samples with continuous and one-hot categorical features and output probability predictions for each class. However, a DLN differs from MLPs in three major ways: it transforms input into binary values at the first step; instead of conducting matrix multiplications between MLP hidden layers, it performs logic operations within the network; and the connections in LogicLayers are very sparse, primarily because each neuron accepts only two inputs.

Our training objectives are twofold: determining which function each neuron should implement and establishing how the neurons should be connected. Neuron function optimization and neuron connection optimization can be divided into two separate tasks because they optimize different sets of parameters. Therefore, we divide the training process into two phases: one for optimizing the internals of neurons (referred to as Phase I) and another for optimizing the connections between neurons (referred to as Phase II). However, both problems are discrete, making common NN training optimizers like gradient descent unsuitable for the task. We have developed methodologies that relax the discrete search spaces to make them continuous and differentiable. We provide details of these methodologies in subsequent subsections; a coarse summary is that we relax discrete search among candidates into a probabilistic mixture of candidates.

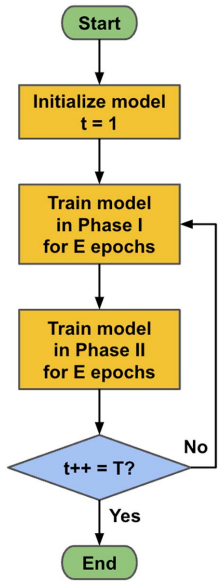


Fig. 2. DLN training flowchart.

We present a high-level overview of the DLN training workflow in Fig. 2 and its algorithmic summary in Algorithm 1. We iteratively update model parameters in the two phases in an alternating manner. After initialization, we conduct Phase I training for E epochs, followed by Phase II training for E epochs. We repeat the combination of Phase I and Phase II training for T iterations. As Algorithm 1 suggests, during one phase, we freeze the trainable parameters of the other phase and perform a hard assignment to its related functions. Thus,

Algorithm 1 Skeleton training process for DLNs

Require: Number of iterations T, epochs per phase E

for $t = 1$ to T **do**
  {Phase I: optimizing neuron functions}
  Quantize and fix neuron connections.
  for epoch $= 1$ to E **do**
    Apply gradient to logic gate distributions for neurons in LogicLayer and to biases and slopes for neurons in ThresholdLayer. {Update neuron parameters}
  end for
  {Phase II: optimizing neuron connections}
  Quantize and fix functions inside neurons.
  for epoch $= 1$ to E **do**
    Apply gradient to find optimal connections between neurons. {Update connection parameters}
  end for
end for

in Phase I, we fix the neuron connections and take argmax over connection parameters; specifically, the link between any two neurons is either 0 or 1 and nothing in between. In Phase II, we freeze and hard-assign the neuron functions so that each neuron performs only one discrete logic operation among all possible operations. After freezing and quantizing the parameters related to the other phase, we relax the discrete parts related to the under-training phase to make optimization differentiable. The algorithm updates relevant parameters solely through gradient descent. Once training is complete, we discretize both the functions and connections of neurons to hard binary states, as Fig. 1 demonstrates. Generally, this discretization leads to a quantization loss; however, this loss is typically small compared to the generalization loss because most components converge well. In some cases, discretization even reduces overfitting and helps with generalization. In our experiments, we set E to 1 and used T as a hyperparameter.

Table II summarizes the trainable parameters for each layer type and their usages in the two training phases and inference. We present the forward computation equation for each layer and, for simplicity, the output function at the neuron level, i.e., the function to calculate the i^{th} output value $\mathbf{y}_i$ from the input vector $\mathbf{x}$. The parameters involved in training are separated by phases and are applied differently between Phase I, Phase II, and inference. For the ThresholdLayer, we use the Heaviside function when its parameters are not under training and use the scaled, shifted Sigmoid function when they are under training. In LogicLayer, we train weights to search for optimal neuron functions and connections. Each weight acts as a learnable probability logit during its training phase to make the network differentiable. During the other phase or inference, the parameter is quantized to its corresponding maximum probability. In SumLayer, weights represent the connection strengths between the neurons of the last hidden layer and the output neurons. They are quantized to binary values during inference.

We present the Phase I training process in Algorithm 2, the Phase II training process in Algorithm 3, and the inference process in Algorithm 4. The processes for the two training phases are quite symmetric because they employ the same procedure; it is just that the layers switch their functions between the two modes. For clear illustration, we employ a mini-batch size of 1 and showcase the layer output computation at the neuron level, i.e., we show how the i^{th} output value is computed. For practical implementation, we use a larger batch size during training and compute the layer output with the help of vectorization. In the two training algorithms, we emphasize the forward pass to highlight details of how to make the forward propagation differentiable. Gradient-based backpropagation is conducted automatically as long as the forward steps are differentiable. During the forward pass, the input sample goes through the DLN, layer by layer; each layer takes the output of the previous layer as its input. Depending on the type of the layer and the phase it is in, each neuron computes its output based on the equations in the pseudo-code, which we discuss in more detail in the following subsections.

A. Logic Operations

Neurons in LogicLayer act as binary logic operators, which are inherently non-differentiable. To overcome this obstacle, we employ real-valued operations during the training phase, as outlined in Table I. This approach effectively softens the discrete logic operations, making them both continuous and differentiable. For inference, we revert to the standard binary $0/1$ logic operations. We use SoftLogic_k to denote the real-valued version of the k^{th} logic operator in Table I and HardLogic_k to denote its binary operation. Consider the example of an AND logic gate, which is described first in the table. For this gate, the soft (real-valued) and hard (binary) outputs are obtained as follows. For the soft output, the real-valued function is:

$$\begin{aligned}\text{real-valued AND}(a,b) &= \text{SoftLogic}_1(a,b)\\ &= a \cdot b,\end{aligned}$$

where $a, b \in [0,1]$ are continuous real numbers. For the hard output, the discretized function is:

$$\begin{aligned}\text{binary AND}(a,b) &= \text{HardLogic}_1(a,b)\\ &= 1_{\{a=1\}} \cdot 1_{\{b=1\}},\end{aligned}$$

where $a, b \in \{0,1\}$ are discrete binary numbers, and indicator functions are applied to them.

To ensure gradient flow between layers, we utilize fuzzy logic expressions throughout the training process, regardless of the phase type. As shown in line 14 of Algorithm 2 and line 14 of Algorithm 3, we use SoftLogic in both training Phase I and Phase II. Line 9 of Algorithm 4 indicates that we use HardLogic for inference.

B. Phase I: Determining Neuron Functions

The left-hand side of Fig. 3 illustrates the process of determining the functions of neurons. In this phase, the connections between neurons are discretized and frozen, allowing us to focus on training the neuron parameters. DLNs consist of three types of layers: ThresholdLayer, LogicLayer, and SumLayer. Correspondingly, there are three types of neurons. Neurons in

TABLE II
SUMMARY OF DLN TRAINABLE PARAMETERS AND FEEDFORWARD FUNCTIONS DURING TRAINING AND INFERENCE. WE USE $\mathbf{x}$ TO DENOTE INPUT AND $\mathbf{y}$ TO DENOTE THE OUTPUT OF EACH LAYER

	ThresholdLayer	LogicLayer	SumLayer
Trainable parameters	Phase I: bias $\mathbf{b} \in \mathbb{R}^{\text{in\_dim}}$ slope $\mathbf{s} \in \mathbb{R}^{\text{in\_dim}}$ Phase II: None	Phase I: logic fn weight $\mathbf{W} \in \mathbb{R}^{\text{out\_dim} \times 16}$ Phase II: link a weight $\mathbf{U} \in \mathbb{R}^{\text{out\_dim} \times \text{in\_dim}}$ link b weight $\mathbf{V} \in \mathbb{R}^{\text{out\_dim} \times \text{in\_dim}}$	Phase I: None Phase II: link weight $\mathbf{S} \in \mathbb{R}^{\text{in\_dim} \times \lvert\mathcal{C}\rvert}$
Training Phase I	$\mathbf{y}_i = \text{Sigmoid}(\mathbf{s}_i \cdot (\mathbf{x}_i - \mathbf{b}_i))$	$\mathbf{y}_i = \sum_{k=0}^{15} \mathbf{P}_k \cdot \text{SoftLogic}_k(\mathbf{x}_{a_i}, \mathbf{x}_{b_i})$ $\mathbf{P} = \text{Softmax}(\mathbf{W}_{i,:})$ $a_i = \arg\max_j \mathbf{U}_{i,j}$ $b_i = \arg\max_j \mathbf{V}_{i,j}$	$\mathbf{y}_i = \sum_{j=0}^{\text{in\_dim}-1} 1_{\{\text{Sigmoid}(\mathbf{S}_{j,i}) \geq \theta_{\text{sum-th}}\}} \cdot \mathbf{x}_j$
Training Phase II	$\mathbf{y}_i = \text{Heaviside}(\mathbf{s}_i \cdot (\mathbf{x}_i - \mathbf{b}_i))$	$\mathbf{y}_i = \text{SoftLogic}_k(a, b)$ $k = \arg\max_j \mathbf{W}_{i,j}$ $a = \sum_{j=0}^{\text{in\_dim}-1} [\text{Softmax}(\mathbf{U}_{i,:})]_j \cdot \mathbf{x}_j$ $b = \sum_{j=0}^{\text{in\_dim}-1} [\text{Softmax}(\mathbf{V}_{i,:})]_j \cdot \mathbf{x}_j$	$\mathbf{y}_i = \sum_{j=0}^{\text{in\_dim}-1} \text{Sigmoid}(\mathbf{S}_{j,i}) \cdot \mathbf{x}_j$
Inference	Same as Phase II	$\mathbf{y}_i = \text{HardLogic}_k(\mathbf{x}_{a_i}, \mathbf{x}_{b_i})$ $k = \arg\max_j \mathbf{W}_{i,j}$ $a_i = \arg\max_j \mathbf{U}_{i,j}$ $b_i = \arg\max_j \mathbf{V}_{i,j}$	Same as Phase I

Algorithm 2 Phase I training

Require: Training dataset $\mathcal{D}$, SumLayer link threshold $\theta_{\text{sum-th}}$
Ensure: Trained model parameters $\mathbf{b}, \mathbf{s}, \mathbf{W}$
Freeze link parameters $\mathbf{U}, \mathbf{V}, \mathbf{S}$
for epoch $= 1, 2, \ldots, E$ **do**
  for all $(\mathbf{x}, z)$ in $\mathcal{D}$ **do**
    {Forward pass}
    for layer in model_layers **do**
      if layer = ThresholdLayer **then**
        {Trainable parameters: $\mathbf{b}, \mathbf{s}$}
        $\mathbf{y}_i = \text{Sigmoid}(\mathbf{s}_i \cdot (\mathbf{x}_i - \mathbf{b}_i))$
      else if layer = LogicLayer **then**
        {Trainable parameters: $\mathbf{W}$}
        $\mathbf{P} = \text{Softmax}(\mathbf{W}_{i,:})$
        $a_i = \arg\max_j \mathbf{U}_{i,j}$
        $b_i = \arg\max_j \mathbf{V}_{i,j}$
        $\mathbf{y}_i = \sum_{k=0}^{15} \mathbf{P}_k \cdot \text{SoftLogic}_k(\mathbf{x}_{a_i}, \mathbf{x}_{b_i})$
      else
        {Trainable parameters: None}
        $\mathbf{y}_i = \sum_{j=0}^{\text{in\_dim}-1} 1_{\{\text{Sigmoid}(\mathbf{S}_{j,i}) \geq \theta_{\text{sum-th}}\}} \cdot \mathbf{x}_j$
      end if
      $\mathbf{x} = \mathbf{y}$ {Take preceding layer's output as input}
    end for
    {Backward pass}
    Compute loss $\mathcal{L}(\mathbf{x}, z)$
    Backpropagate to compute gradients
    Update neuron function parameters $\mathbf{b}, \mathbf{s}, \mathbf{W}$
  end for
end for

Algorithm 3 Phase II training

Require: Training dataset $\mathcal{D}$
Ensure: Trained model parameters $\mathbf{U}, \mathbf{V}, \mathbf{S}$
Freeze neuron function parameters $\mathbf{b}, \mathbf{s}, \mathbf{W}$
for epoch $= 1, 2, \ldots, E$ **do**
  for all $(\mathbf{x}, z)$ in $\mathcal{D}$ **do**
    {Forward pass}
    for layer in model_layers **do**
      if layer = ThresholdLayer **then**
        {Trainable parameters: None}
        $\mathbf{y}_i = \text{Heaviside}(\mathbf{s}_i \cdot (\mathbf{x}_i - \mathbf{b}_i))$
      else if layer = LogicLayer **then**
        {Trainable parameters: $\mathbf{U}, \mathbf{V}$}
        $k = \arg\max_j \mathbf{W}_{i,j}$
        $a = \sum_{j=0}^{\text{in\_dim}-1} [\text{Softmax}(\mathbf{U}_{i,:})]_j \cdot \mathbf{x}_j$
        $b = \sum_{j=0}^{\text{in\_dim}-1} [\text{Softmax}(\mathbf{V}_{i,:})]_j \cdot \mathbf{x}_j$
        $\mathbf{y}_i = \text{SoftLogic}_k(a, b)$
      else
        {Trainable parameters: $\mathbf{S}$}
        $\mathbf{y}_i = \sum_{j=0}^{\text{in\_dim}-1} \text{Sigmoid}(\mathbf{S}_{j,i}) \cdot \mathbf{x}_j$
      end if
      $\mathbf{x} = \mathbf{y}$ {Take preceding layer's output as input}
    end for
    {Backward pass}
    Compute loss $\mathcal{L}(\mathbf{x}, z)$
    Backpropagate to compute gradients
    Update link parameters $\mathbf{U}, \mathbf{V}, \mathbf{S}$
  end for
end for

the SumLayer simply aggregate the outputs from the preceding LogicLayer that connect to them; therefore, they do not require any parameters to describe their function. During this phase, we focus on training the parameters for neurons in ThresholdLayer and LogicLayer. In the following subsections, we detail the training algorithms for each layer type.

Algorithm 4 Inference

Require: Input data $\mathbf{x}$

Require: Model layer list $\begin{bmatrix} \text{ThresholdLayer} \\ \text{LogicLayer}_1 \\ \dots \\ \text{LogicLayer}_n \\ \text{SumLayer} \end{bmatrix}$

1: {First layer: ThresholdLayer}
2: $\mathbf{y}_i = \text{Heaviside}(\mathbf{s}_i \cdot (\mathbf{x}_i - \mathbf{b}_i))$
3: {Middle layers: LogicLayer}
4: **for** $l = 1, 2, \dots, n$ **do**
5: $k = \arg\max_j \mathbf{W}_{i,j}$
6: $a_i = \arg\max_j \mathbf{U}_{i,j}$
7: $b_i = \arg\max_j \mathbf{V}_{i,j}$
8: $\mathbf{x} = \mathbf{y}$ {Take preceding layer's output as input}
9: $\mathbf{y}_i = \text{HardLogic}_k(\mathbf{x}_{a_i}, \mathbf{x}_{b_i})$
10: **end for**
11: {Last layer: SumLayer}
12: $\mathbf{z}_i = \sum_{j=0}^{\text{in\_dim}-1} 1_{\{\text{Sigmoid}(\mathbf{S}_{j,i}) \geq \theta_{\text{sum-th}}\}} \cdot \mathbf{y}_j$
13: {Get predicted label}
14: $\hat{y} = \arg\max_i \mathbf{z}_i$
15: **return** $\hat{y}$

1) ThresholdLayer: The purpose of the ThresholdLayer is to binarize continuous variables using threshold functions. This layer takes a continuous vector as input and outputs a binary vector. Each neuron takes one scalar input and outputs a scalar value. The output binary vector can later be concatenated with one-hot encoded categorical feature values to form the final binary vector for the next layer. We connect each continuous input value to multiple neurons to map it to a finite set of values, an approach similar to binning but not limited to mutually exclusive bins. Each neuron takes in one scalar and serves as a threshold function with trainable bias and scale. During training, we soften the non-differentiable threshold function by substituting it with a shifted and scaled sigmoid function:

$$\begin{aligned} f(x) &= \text{Sigmoid}(s \cdot (x - b)) \\ &= \frac{1}{1 + e^{-[s \cdot (x-b)]}}, \end{aligned}$$

where b represents the cutoff and s denotes the slope of the input passed to the Sigmoid function. This modification enables gradient flow to both b and s. During Phase I of training, both b and s are trainable. During Phase II or inference, we quantize the soft threshold function to the normal threshold function:

$$F(x) = \text{Heaviside}(s \cdot (x - b))$$

and freeze parameters b and s. In Fig. 3, we illustrate the training mode of the ThresholdLayer on the left and the inference mode on the right (ThresholdLayer functions the same way in both Phase II and during inference). In Phase I, the highlighted neuron applies a shifted and scaled Sigmoid function to its input. After forward and backward propagations, gradients suggest a positive increase in s (which results in steeper scaling) and a positive increase in b (which results in a rightward shift of the threshold function). When in Phase II, the function of this neuron is quantized and fixed to a Heaviside function and is no longer trainable. We show the Phase I training pseudo-code for the ThresholdLayer in lines 6 to 8 in Algorithm 2, the Phase II training pseudo-code in lines 6 to 8 in Algorithm 3, and the inference pseudo-code in line 2 in Algorithm 4.

In our experiments, we preprocess the input data by applying Min-Max Scaling to continuous variables and one-hot encoding to categorical variables, ensuring that all input values range between 0 and 1. We allocate four or six neurons to each continuous input and initialize the starting slope to 2, a value not too flat but still smooth in the beginning. Inspired by Gorishniy et al. [29] who apply decision tree-based binning to transform the scalar continuous features to vectors, we initialize b for each neuron by the bin edges of calculated decision trees. Hence, we fit a decision tree using training data, by specifying the maximum number of leaf nodes. We have observed that most neurons in the ThresholdLayer converge during training, with some converging to values within the 0 and 1 range while others do not. Neurons whose biases converge to below 0 or over 1 act as constant Boolean False or True. We found that out-of-range convergence leads to good feature selection. As we show in Sec. IV-E, DLNs with fixed (i.e., non-trainable) ThresholdLayers underperform DLNs with trainable ThresholdLayers and are larger in size. Fig. 4 shows an example of a neuron's subsampled training process. It plots the resulting neuron function based on the b and s values at the sampled epochs. In the beginning, it starts with a certain cutoff and a small slope, thus initially acting as a smooth function. As the epochs progress, both the cutoff and slope converge, with the bias shifting to the left and the slope increasing, making the neuron increasingly resemble a threshold function. We provide an illustration of the training process of one continuous feature in Fig. 5. In this example, we connect the feature to four threshold neurons. Inside the neurons, all slopes are initialized to 5, and the initial bias values are $[0.2, 0.4, 0.7, 0.85]$, calculated by a decision tree. After training, slopes converge to larger values, making the soft threshold functions steeper; biases moved to $[-0.2, 0.3, 0.8, 1.3]$, with some still within the range of the feature and some outside. During inference, discretized threshold functions are adopted.

2) LogicLayer: As indicated in Table I, a two-input neuron can implement $2^{2^2} = 16$ Boolean functions. Next, our goal is to determine which of the 16 classes each neuron should belong to. This can be treated as a classification problem. We employ the method introduced in the DDLGN article [1]. During training, each neuron is considered a weighted sum of all 16 operators. This weight is trainable. To make sure that the weights associated with the operators sum to 1, a Softmax function is applied to the weights. Let a 16-valued weight vector $\mathbf{w}$ denote the weights for a sample logic neuron. Thus, $\mathbf{w}_i$ corresponds to the i^{th} Boolean operator. In this phase, connections to neurons are fixed, with each logic neuron receiving inputs from exactly two

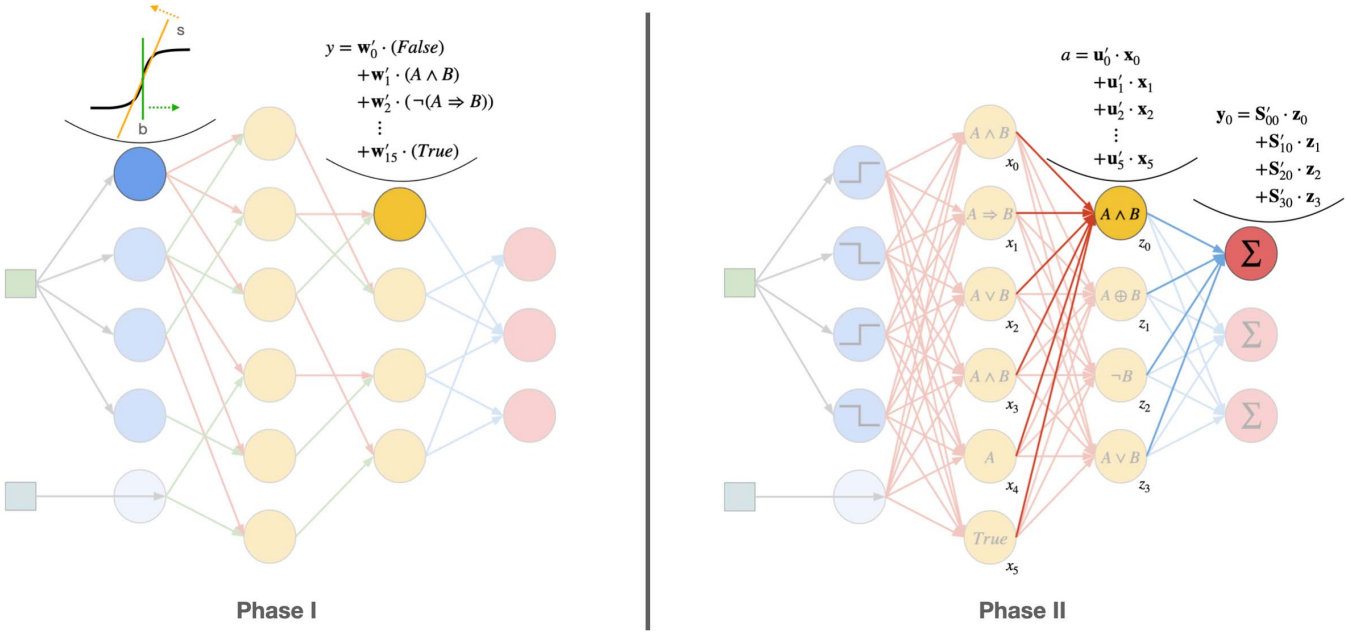


Fig. 3. Illustration of the two-phase training algorithm. During Phase I, we discretize and fix the connections between neurons while training their parameters within ThresholdLayer and LogicLayer. In Phase II, we hold the neuron operations constant and focus on optimizing the connections between them. Key details are highlighted for clarification.

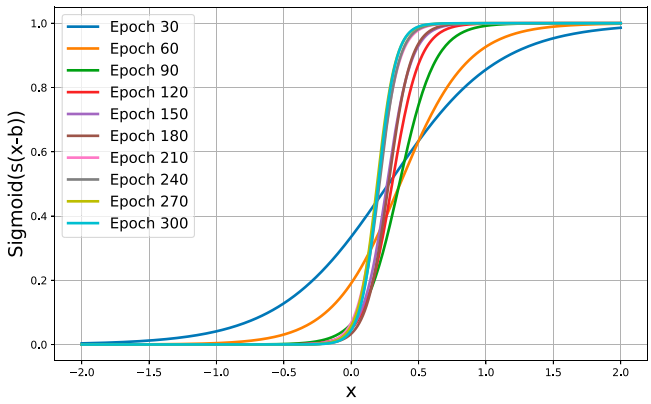


Fig. 4. Example training process of a ThresholdLayer neuron

.

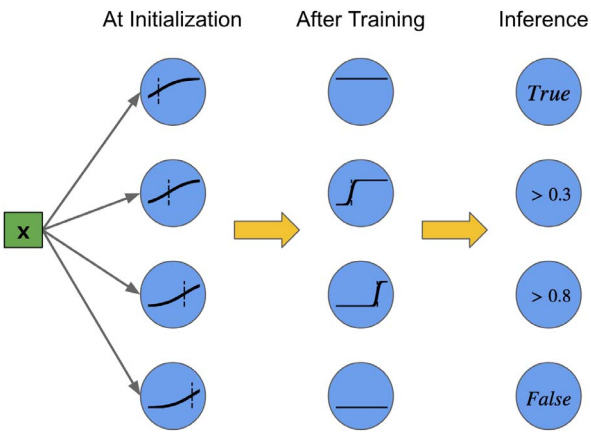


Fig. 5. Illustration of the ThresholdLayer training process for a continuous feature.

input neurons. Given scalar inputs a and b, the output y of the logic neuron is given by the sum of logic operator mixtures:

$$y = \sum_{k=0}^{15} [\text{Softmax}(\mathbf{w})]_k \cdot \text{SoftLogic}_k(a, b)$$
$$= \sum_{k=0}^{15} \frac{e^{\mathbf{w}_k}}{\sum_j e^{\mathbf{w}_j}} \cdot \text{SoftLogic}_k(a, b).$$

Lines 11 to 14 of Algorithm 2 calculate the weights and sum up the 16 logic function outputs.

During Phase II or inference, we quantize the summation by selecting the one operator that has the largest corresponding weight. We find the index of this operator and employ solely its operation as the logic neuron's function:

$$k = \arg\max_j \mathbf{w}_j,$$
$$y = \begin{cases} \text{SoftLogic}_k(a, b), & \text{if training,} \\ \text{HardLogic}_k(a, b), & \text{otherwise.} \end{cases}$$

In Algorithm 3, line 11 finds the index, and line 14 performs the real-valued logic operation. In the inference algorithm, Algorithm 4, line 5 finds the index, and line 9 performs the binary logic operation. We illustrate in Fig. 3 the training mode of the LogicLayer neuron function on the left and the inference mode on the right. Neuron function training only occurs in Phase I; hence, neurons perform the same function in Phase II and during inference. In Phase I, the output of a neuron is a weighted sum of all 16 logic functions, and this weight is trainable. When not under training, the function of the neuron is frozen and attached to one type of operator.

In practice, with the aid of the search for neuron connections in Phase II, we find that most neurons can converge to

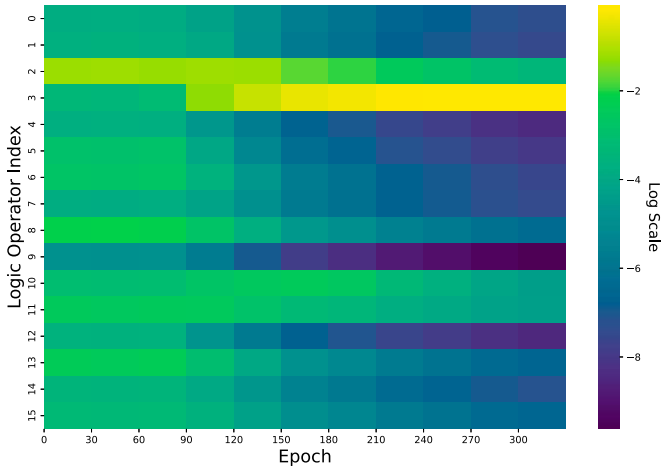


Fig. 6. Example training of a LogicLayer neuron. Note that colors are displayed on a logarithmic scale.

a single operator. In Fig. 6, we show a typical example of a logic neuron's subsampled training process. The x-axis displays the epoch number and the y-axis shows the softmaxed weight distribution of the 16 operators on a logarithmic color scale. As illustrated in the figure, this neuron rapidly converges to the fourth (index no. 3, shown in yellow) logic operator.

C. Phase II: Determining Connections

The right-hand side of Fig. 3 illustrates the training process for determining the connections between neurons. In this phase, we fix the functions of neurons; that is, the parameters of the threshold functions for threshold neurons and the selection of logic operators for logic neurons. For the ThresholdLayer, the input connections are fixed after initialization. In our setup, each continuous input feature is connected to several threshold neurons, and each neuron takes only one input feature. Therefore, the ThresholdLayer does not have trainable parameters in Phase II. Next, we describe how connections to the LogicLayers and SumLayer are determined.

1) LogicLayer: Each neuron in a LogicLayer has two inputs, denoted as a and b. The objective is to determine which two input values, among in_dim input values from the previous layer's output, should be linked to a and b. We divide this task into two sub-goals: determining the source of a and determining the source of b. During training, we soften the connections by representing them as weighted sums of all inputs. We showcase our method for finding the first input, a, to the logic neuron. The method for finding a connection to b is the same. Let $\mathbf{x} \in \mathbb{R}^{\text{in\_dim}}$ represent the input of the logic neuron under study. In a LogicLayer, there are out_dim neurons; suppose we pick a random neuron for study. By design, the input a of this neuron must be one of the elements in $\mathbf{x}$. Hence, the goal is to find the index of that element. This is a discrete optimization problem. During Phase II training, we relax the problem through weighted connections to all elements in $\mathbf{x}$ and reduce the problem to training a weight vector $\mathbf{u} \in \mathbb{R}^{\text{in\_dim}}$ to learn the weights linking the logic neuron to its input. We use a Softmax function to make sure that the weights sum to 1. The equation for the relaxed a is then a probabilistic mixture of $\mathbf{x}$:

$$a = \sum_{j=0}^{\text{in\_dim}-1} [\text{Softmax}(\mathbf{u})]_j \cdot \mathbf{x}_j = \sum_{j=0}^{\text{in\_dim}-1} \frac{e^{\mathbf{u}_j}}{\sum_i e^{\mathbf{u}_i}} \cdot \mathbf{x}_j.$$

This approach allows each neuron to be non-uniformly connected to all neurons from the preceding layer during training. In practice, the neuron learns the importance of incoming connections and typically converges to the most relevant one.

During Phase I or inference, the connections are quantized by selecting the most heavily weighted input neuron, resulting in

$$k = \arg\max_j \mathbf{u}_j,$$
$$a = \mathbf{x}_k.$$

The left side of Fig. 3 illustrates fixed connections to logic neurons, with a colored in red and b colored in green. Each neuron takes only one input as a and one as b. Connections are discretized and fixed during both Phase I training and inference. Lines 12 and 13 in Algorithm 2 show discrete link selection during Phase I training. Lines 6 and 7 in Algorithm 4 show the same argmax selections during inference. On the right side of Fig. 3, we highlight an example of a connection weight matrix during training. In this phase, the neuron's incoming a is connected to all its inputs, and the value of the final a is a weighted sum of all these candidates. We expect gradient descent steps to make the weight concentrate on one link in the end. Lines 12 and 13 in Algorithm 3 show the relaxed, differentiable link search for logic neurons during Phase II training.

2) SumLayer: The SumLayer has a number of neurons equal to the number of classes, $|\mathcal{C}|$, with each neuron representing a class. This setup is similar to the output layer of an MLP. Each neuron in the SumLayer connects to a subset of neurons from the previous layer and sums up their output scores. These scores are normalized, then used as logits, and later passed to the Cross-Entropy Loss function for training. To maximize the expressiveness of the network, we allow nodes in the last hidden layer to connect to multiple neurons in the SumLayer. Consequently, this setup translates to a classification problem for each (input, output) neuron pair: determine whether they should connect or not. We train a matrix $\mathbf{S} \in \mathbb{R}^{\text{in\_dim} \times |\mathcal{C}|}$ and treat $\text{Sigmoid}(\mathbf{S}_{j,i})$ as the probability that the j^{th} input neuron should connect with the i^{th} output class. In this training phase, for any output neuron, we sum up probabilistic connections from all inputs to enable differentiability, as illustrated on the right side of Fig. 3.

Suppose we pick the i^{th} output neuron to study. Mathematically, we compute its output $\mathbf{y}_i$ from input $\mathbf{x} \in \mathbb{R}^{\text{in\_dim}}$ using:

$$\mathbf{y}_i = \sum_{j=0}^{\text{in\_dim}-1} \text{Sigmoid}(\mathbf{S}_{j,i}) \cdot \mathbf{x}_j.$$

Line 17 of Algorithm 3 implements this equation.

When not in training mode, i.e., in Phase I or in inference, the output logit $\mathbf{y}_i$ is the binary sum of inputs that have connection

probability above a threshold, namely $\theta_{\text{sum-th}}$, as illustrated on the left side of Fig. 3. Mathematically,

$$\mathbf{y}_i = \sum_{j=0}^{\text{in\_dim}-1} 1_{\{\text{Sigmoid}(\mathbf{S}_{j,i}) \geq \theta_{\text{sum-th}}\}} \cdot \mathbf{x}_j.$$

We show the pseudo-code for this equation in line 17 of Algorithm 2 and in line 12 of Algorithm 4. We impose binary connections to SumLayers to maximize interpretability. We found that setting $\theta_{\text{sum-th}}$ to 0.8 works well, in general.

D. Searching Over Subspaces

In LogicLayers, for each neuron, the full search space has a size of 16 for the neuron type and a size of in_dim for the incoming link. However, full search is often not necessary. For example, any logic circuit can be constructed using just NAND gates. Selecting a subset of gates simplifies the training objective and helps with the vanishing gradient problem, as it reduces the weight dimension in Softmax layers. We could select only logic gates that exhibit good gradient flow. From Table I, we can see that the OR gate has better gradient flow than the AND gate because inputs A and B are between 0 and 1. A similar situation applies to link search: there are many paths in a network that can yield good local optima. Fortunately, searching over a subspace can be flexibly implemented, thanks to the properties of the Softmax function. We just need to assign $-\infty$ to the weight entries that we want to turn off the search for; their Softmax probabilities will then be zero. Thus, they will never be active during training and inference, and there is no gradient flow to them.

We sort the gate types by logic completeness and gradient flow, from high to low: NOR, NAND, XOR, XNOR, OR, AND, A, B, NOT A, NOT B, B IMPLIES A, A IMPLIES B, NOT (A IMPLIES B), NOT (B IMPLIES A), False, True. We can just include the first k candidates from the list and mask out the others. To obtain a subset of the link space, we randomly select k incoming candidates and mask out the weights of the other candidates by assigning them $-\infty$. In practice, we found that using a subset of 8 logic operators and a subset of 8 links works the best. Thus, we use this setting as the default. We present the results of not using subsets, using a subset size of 16 for link search, and using a subset size of 4 for both gates and links search in Sec. IV-E.

E. Using Straight-Through Estimators

By design, every layer in a DLN employs either a Sigmoid or Softmax function. This enables differentiability of the network. However, although Sigmoid and Softmax functions soften the step and argmax functions, they also weaken output activations. After encountering layers of Sigmoid and Softmax functions, the layer outputs tend towards the average and become more uniform, making the gradient descent method harder to work with. To sharpen the function outputs while enabling backpropagation of gradients at the same time, we employ the STE. During the forward pass, STE applies the non-differentiable function (e.g., threshold, argmax, etc.) normally, but during the backward pass, it bypasses the non-differentiable function by treating its derivative as 0, allowing gradients from the subsequent layer to pass through unchanged. Mathematically, assume f is the discretized output value and g is the soft value; the final output h then is

$$h = \text{detach}(f - g) + g,$$

where detach is an operator that detaches the value from the backpropagation graph. Thus, the value f is passed to h but only g is counted for calculating gradients. This enables the output to attain the same value as the discrete function during the forward pass, and the same gradient as the soft function during the backward pass.

In practice, we find STE useful for DLN training and have set its use as the default. We provide results for the case when STE is not used in Sec. IV-E.

F. Concatenating Inputs

The Wide & Deep Model [20] work demonstrates the effectiveness of bringing some inputs closer to the output layer. We adopt this method by concatenating the ThresholdLayer to every intermediate LogicLayer, i.e., from the second layer to the second last. The concatenated ThresholdLayer may either be the same layer reused at multiple places or different ThresholdLayers in each place. Experimental results show that using different input layers is better than using the same layer, and both approaches are better than not concatenating. We set multiple input concatenations as the default and present the results of other mentioned methods in Sec. IV-E.

G. Model Simplification

After training, we obtain logic expressions from the network and simplify them using SimPy [30]. We have found that this step is generally beneficial for reducing model size because our trainable ThresholdLayer acts as a feature selector. We also explore other simplification methods such as pruning and present the results in the ablation studies section, Sec. IV-E. Based on the results, we find that SimPy is sufficient for model simplification.

IV. Experiments

We test our method and compare it with seven other methods on 20 tabular datasets. The seven methods we test are k-nearest neighbors (KNN), decision tree (DT), random forest (RF), Gaussian Naive Bayes (NB), AdaBoost (AB), MLP, and tree-initialized DDLGN (abbreviated to LGN in our tables). We list the characteristics of the datasets after preprocessing in Table III. Their sample sizes range from hundreds to thousands, and their number of classes ranges from two to 26, which is typical for tabular datasets.

A. Experimental Setup

We first preprocess the datasets. This involves removing samples with too many missing values, one-hot encoding categorical features, and scaling continuous features using MinMax scalers to ensure they range between 0 and 1. For each

TABLE III
CHARACTERISTICS OF THE POST-PROCESSED DATASETS AND THE AVERAGE BALANCED-CLASS TEST ACCURACY PERCENTAGE OF MODELS

Dataset	# train	# test	# class	# cont.	# cate.	KNN	DT	RF	NB	AB	MLP	LGN	DLN
BankChurn	7500	2500	2	6	5	65.1	69.5	71.6	64.4	70.7	74.9	75.6	75.8
BreastCancer	214	72	2	0	34	61.4	62.1	66.1	57.1	56.6	60.2	62.4	65.1
Cardiotocography	1594	532	3	19	22	97.5	95.9	96.0	96.1	87.8	97.2	97.5	97.7
CKD	300	100	2	12	9	99.2	96.7	97.8	96.6	100	99.4	99.5	99.1
Cirrhosis	203	68	3	10	5	46.8	50.7	51.5	45.7	46.9	50.9	47.5	54.0
Compas	3958	1320	2	5	8	65.5	63.7	64.9	62.5	68.1	66.6	67.1	67.6
CreditFraud	1107	369	2	30	0	93.3	92.0	93.1	91.0	94.0	93.0	94.1	94.3
Diabetes	292	98	2	13	1	71.3	85.6	86.1	80.8	80.2	78.7	83.0	88.0
EyeMovements	8202	2734	3	20	3	58.1	44.3	45.4	45.2	53.9	52.1	54.2	55.0
HAR	7352	2947	6	63	0	78.1	70.3	75.6	81.4	72.4	90.8	81.8	87.1
Heart	219	74	2	5	14	80.1	71.8	81.0	83.2	80.5	80.2	74.6	81.5
HeartFailure	224	75	2	6	5	50.9	65.2	68.6	65.3	64.4	59.7	62.6	68.4
HepatitisC	461	154	2	11	1	86.6	90.7	93.2	88.8	93.9	91.3	86.8	91.1
Letter	15000	5000	26	16	0	94.9	39.7	33.6	64.1	41.2	94.7	42.2	69.1
Liver	434	145	2	9	1	57.4	63.9	66.2	69.2	55.5	69.1	66.5	66.5
Mushroom	5898	1966	2	0	67	100	97.6	98.6	97.9	100	100	100	100
PDAC	516	172	2	31	0	49.5	52.6	60.9	62.1	57.7	58.4	54.4	59.6
SatIm	4435	2000	6	36	0	89.1	79.5	81.9	78.9	71.9	89.0	84.9	86.4
Stroke	3831	1278	2	3	11	52.4	73.4	75.9	69.8	50.5	62.3	73.4	76.2
TelcoChurn	5262	1754	2	2	21	70.2	73.3	75.0	74.6	70.9	68.7	74.5	75.9
Average Rank						5.2	6.1	4.0	5.3	5.1	4.2	3.9	2.3

(dataset, model) pair, we conduct four experiments, each with a different random seed. Each experiment consists of three steps: hyperparameter search, training, and evaluation. We sample 32 parameter sets for every hyperparameter search and select the set with the best cross-validated balanced-class test accuracy. The number of folds ranges from two to four, with datasets containing fewer samples having more folds. We employ the Optuna search algorithm [31] for neural network-based models (i.e., MLP, DDLGN, DLN) and use random sampling for traditional methods (i.e., KNN, DT, RF, NB, AB). We use the scikit-learn package [32] for data processing and experiments with traditional methods. We train NN models using PyTorch [33]. We use Ray [34] to run parallel hyperparameter searches.

B. Accuracy

We measure the predictive power of the models using balanced-class accuracy. Table III presents the average results. DLN achieved the best average rank of 2.3 and NN-based methods generally outperform traditional methods. Another observation is that bagging and boosting help, as random forest and AdaBoost outperform the decision tree, as expected. DDLGN requires binary inputs; hence, continuous features need to be binned before passing to the network. In the original work [1], DDLGN bins are selected manually for tabular data and uniformly for image data. We found that, compared to uniform thresholds, tree-based binning [29] improves DDLGN accuracy by a significant margin. Thus, we present results for tree-initialized DDLGNs.

C. Efficiency

We measure efficiency from two perspectives: inference cost and disk space cost. For inference computation cost, we provide the number of high-level operations (OPs) and the corresponding number of basic logic gate-level operations in Table IV. High-level operations include addition, comparison, multiplication, division, logarithm, etc. For each high-level operation, depending on the input type (e.g., float32, int64), we can compute its total number of operations in basic hardware logic gates. For example, to perform addition of two n-bit inputs, the least significant bit (LSB) is fed to a half-adder and the other bits to full-adders. A half-adder can be implemented with 5 NAND gates and a full-adder with 9 NAND gates. Hence, adding two n-bit inputs requires $5 + 9 * (n - 1)$ NAND operations. We can similarly convert all high-level operations to basic 2-input logic operations. We count AND/OR/NAND/NOR as one logic operation, XOR/XNOR as three, and NOT as 0. We used the default number types when training models, i.e., float32 for NNs and float64 for most traditional models. Since most models can maintain inference accuracy even after downgrading the datatype to float16, we assume that floating-point numbers have a data type of float16 and integers have a data type of int16 when presenting the counts in Table IV. DLN would have a higher advantage with larger datatypes because the only place it needs floating-point operations is when comparing continuous features with threshold values in the ThresholdLayer. Thus, this advantage of DLN is muted in Table IV. The table shows that decision trees are the most computationally efficient method and DLN is the second best. Logic-based networks are orders of magnitude more efficient than multiplication-based networks.

DLNs can be implemented on an ASIC or FPGA easily because they only consist of comparators, logic operators, and aggregators. They also have an advantage in inference speed and energy efficiency. Take MLPs as an example for comparison. From Table IV, the geometric mean of the number of high-level OPs for MLPs is 3.83K, and the majority of them are FLOPs. If we assume an edge device with 10 GFLOPS, then, in an ideal case that does not take into account the forward propagation

TABLE IV
AVERAGE NUMBER OF OPERATIONS FOR INFERENCE, ASSUMING FLOAT16 FOR FLOATING-POINT AND INT16 FOR INTEGER

	High-level OPs								Basic logic gate-level OPs							
	KNN	DT	RF	NB	AB	MLP	LGN	DLN	KNN	DT	RF	NB	AB	MLP	LGN	DLN
Bank	316K	2	515	72	300	1.1K	251	52	106M	366	68.3K	323K	143K	696K	3.8K	1.9K
Breast	21.5K	3	578	210	378	6.7K	126	61	8.2M	503	81.2K	386K	180K	4.1M	545	256
Cardio	162K	5	836	378	478	10.3K	721	211	37.4M	915	129K	608K	227K	6.3M	10.6K	6.1K
CKD	21.1K	3	614	132	426	3.2K	351	91	9.0M	412	86.2K	350K	203K	1.9M	7.4K	4.4K
Cirrhosis	9.0K	3	157	144	389	1.7K	324	120	2.7M	412	7.8K	501K	185K	1.0M	5.6K	6.0K
Compas	185K	2	404	84	419	1.1K	179	60	64.1M	229	48.0K	328K	200K	647K	2.8K	2.7K
Credit	76.5K	2	636	186	437	5.9K	755	169	10.1M	366	87.5K	375K	208K	3.6M	17.0K	10.5K
Diabetes	14.4K	1	150	90	328	1.1K	369	44	6.0M	183	10.5K	331K	156K	667K	5.8K	3.2K
EyeMove	476K	2	417	216	478	9.6K	545	297	65.3M	366	51.8K	534K	227K	5.9M	10.7K	10.2K
HAR	1.4M	4	730	1.1K	542	40.5K	1.9K	392	520M	686	108K	1.4M	258K	25.1M	29.2K	31.4K
Heart	10.8K	3	394	120	457	1.9K	167	80	2.5M	412	48.3K	345K	218K	1.2M	2.8K	1.3K
HeartFail	7.7K	3	189	72	376	754	149	48	2.3M	458	13.7K	323K	179K	458K	2.9K	2.4K
Hepatitis	14.7K	3	486	78	527	1.7K	190	57	2.1M	549	66.7K	326K	251K	1.1M	6.2K	4.1K
Letter	853K	6	524	1.3K	523	15.6K	599	189	297M	1.0K	71.4K	4.4M	249K	9.6M	8.7K	22.4K
Liver	14.2K	2	179	66	438	606	192	39	4.2M	366	13.5K	320K	209K	366K	4.6K	3.2K
Mush.	1.1M	3	779	408	509	25.5K	350	320	332M	458	117K	476K	242K	15.8M	1.2K	1.5K
PDAC	52.1K	1	130	192	368	7.4K	496	59	22.7M	137	3.0K	378K	175K	4.6M	18.9K	5.0K
SatIm	448K	6	880	666	435	14.6K	858	422	138M	1.0K	135K	1.2M	207K	9.0M	16.2K	25.9K
Stroke	176K	2	612	90	522	1.9K	194	40	51.2M	229	86.1K	331K	249K	1.1M	1.9K	1.6K
Telco	362K	3	551	144	384	3.3K	182	67	109M	503	74.8K	356K	183K	2.0M	1.6K	1.3K
Rank	8.0	1.0	5.0	3.4	4.9	7.0	4.7	2.1	8.0	1.1	3.9	6.0	5.0	7.0	2.8	2.3

TABLE V
AVERAGE NUMBER OF MODEL PARAMETERS AND DISK SPACE IN BYTES, ASSUMING FLOAT16 FOR NUMERICAL VALUES AND INT16 FOR INDICES

	Number of parameters								Disk space in Bytes							
	KNN	DT	RF	NB	AB	MLP	LGN	DLN	KNN	DT	RF	NB	AB	MLP	LGN	DLN
Bank	90.0K	13	3.0K	46	702	598	586	129	180K	25	5.9K	92	1.4K	1.2K	1.2K	258
Breast	7.5K	16	3.6K	138	886	3.4K	271	156	15.0K	32	7.2K	276	1.8K	6.8K	542	313
Cardio	66.9K	29	5.2K	249	1.1K	5.2K	1.7K	582	134K	57	10.3K	498	2.2K	10.4K	3.4K	1.2K
CKD	6.6K	14	3.1K	86	998	1.6K	834	232	13.2K	27	6.1K	172	2.0K	3.2K	1.7K	465
Cirrhosis	3.2K	24	290	93	910	880	767	298	6.5K	47	580	186	1.8K	1.8K	1.5K	596
Compas	55.4K	8	1.8K	54	980	554	414	136	111K	15	3.6K	108	2.0K	1.1K	828	271
Credit	34.3K	14	3.8K	122	1.0K	3.0K	1.8K	413	68.6K	27	7.5K	244	2.0K	6.0K	3.7K	826
Diabetes	4.4K	6	354	58	766	568	872	99	8.8K	12	708	116	1.5K	1.1K	1.7K	196
EyeMove	197K	11	1.9K	141	1.1K	4.8K	1.3K	989	394K	22	3.7K	282	2.2K	9.7K	2.6K	2.0K
HAR	471K	39	6.2K	762	1.3K	20.4K	4.5K	1.1K	941K	77	12.5K	1.5K	2.5K	40.8K	9.1K	2.3K
Heart	4.4K	19	1.6K	78	1.1K	978	391	209	8.8K	37	3.2K	156	2.1K	2.0K	782	418
HeartFail	2.7K	14	451	46	878	392	354	111	5.4K	27	902	92	1.8K	784	708	222
Hepatitis	6.0K	21	2.7K	50	1.2K	900	475	127	12.0K	42	5.4K	100	2.5K	1.8K	950	254
Letter	255K	101	6.1K	858	1.2K	7.9K	1.4K	1.3K	510K	202	12.2K	1.7K	2.5K	15.9K	2.7K	2.6K
Liver	4.8K	15	440	42	1.0K	318	468	98	9.6K	30	880	84	2.0K	635	936	196
Mush.	401K	18	4.9K	270	1.2K	12.9K	766	1.3K	802K	35	9.8K	540	2.4K	25.7K	1.5K	2.5K
PDAC	16.5K	5	158	126	861	3.7K	1.3K	131	33.0K	10	316	252	1.7K	7.5K	2.5K	262
SatIm	164K	48	6.4K	438	1.0K	7.4K	2.0K	1.2K	328K	95	12.8K	876	2.0K	14.8K	4.1K	2.5K
Stroke	57.5K	8	3.6K	58	1.2K	964	444	99	115K	15	7.3K	116	2.4K	1.9K	888	197
Telco	126K	18	3.0K	94	900	1.7K	410	167	253K	35	6.1K	188	1.8K	3.4K	820	334
Rank	8.0	1.0	6.1	2.0	5.2	5.9	4.7	3.3	8.0	1.0	6.1	2.0	5.2	5.9	4.7	3.3

delays, it would take an average of 383 ns to perform inference. For DLNs, even without considering the fact that the majority of the OPs are binary, the average would be 37 times smaller: 10 ns. To estimate the energy consumption, we can use the number of basic logic gate operations. The geometric means for MLPs and DLNs are 2.35M and 3.94K, respectively. For devices that need 10 pJ per basic logic operation, MLPs require 23.5 μJ while DLNs only need about 600 times less, i.e., 39.4 nJ.

Table V presents the number of parameters and the required disk space for storage. Trees need to store threshold values and indices of features and child nodes. DLNs need to store the functionality of neurons and indices of their incoming links. We assumed that each float/int requires 16 bits and each index also needs 16 bits. On an average, decision trees are the smallest in size and naive Bayes the second smallest. The average rank of DLNs is 3.3, which is the smallest among NNs and substantially smaller than vanilla MLPs.

D. Interpretability

DLNs are interpretable by nature and their interpretability is reinforced by the feature selection power of ThresholdLayers. Figs. 7–9 illustrate the decision-making process of DLNs. Take the DLN in Fig. 7 as an example. It achieves a balanced-class test accuracy of 59.3% on the Cirrhosis dataset and makes decisions based on 6 continuous features and 4 categorical

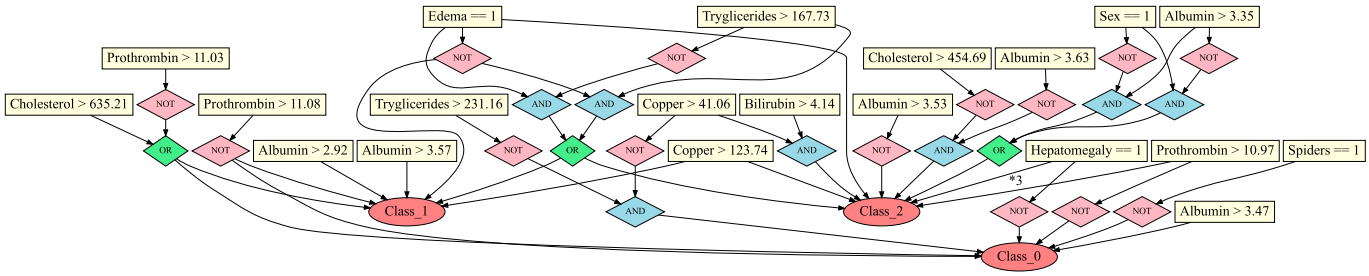


Fig. 7. Visualization of the decision-making process of a DLN for predicting cirrhosis disease. It achieves a balanced-class test accuracy of 59.3%. The DLN uses 6 continuous features and 4 categorical features, selected from an original set of 10 continuous features and 5 categorical features.

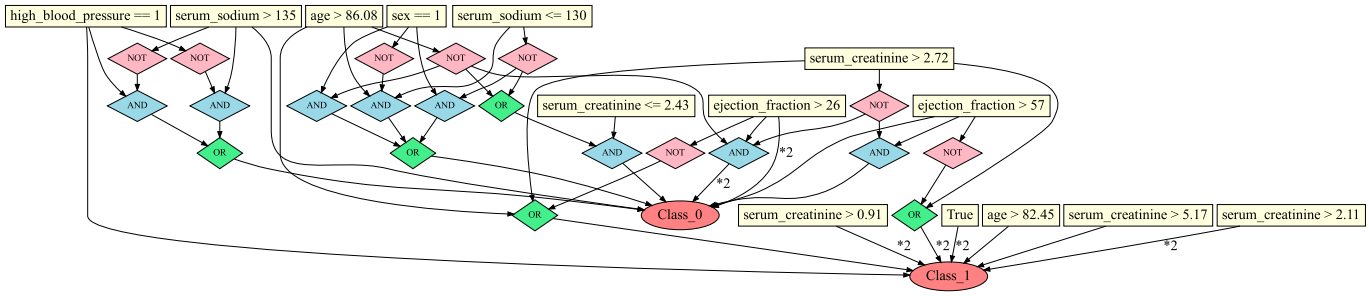


Fig. 8. Visualization of the decision-making process of a DLN for predicting heart failure disease. It achieves a balanced-class test accuracy of 70.5%. The DLN uses 4 continuous features and 2 categorical features, selected from an original set of 6 continuous features and 5 categorical features.

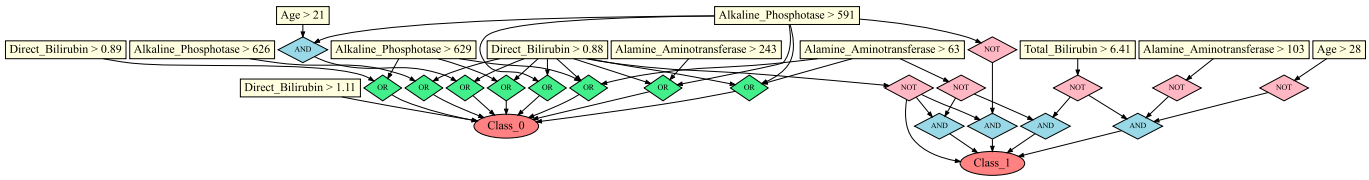


Fig. 9. Visualization of the decision-making process of a DLN for predicting liver disease. It achieves a balanced-class test accuracy of 72.5%. The DLN uses 5 continuous features, selected from an original set of 9 continuous features and 1 categorical feature.

features, chosen from 10 continuous and 5 categorical features. Class_0, Class_1, and Class_2 represent the three class types. Yellow boxes represent features and diamonds represent logic operators. The DLN binarizes inputs based on comparisons for continuous features and equality checks for categorical features. Logic operators receive binary inputs and output binary values. Class nodes receive binary scores from either feature boxes or logic operators and aggregate the scores. A link to a class node represents a logic rule. The multiplier $*$ on the right of the link to the class node represents the strength of the associated rule. The class with the highest score is the final prediction.

E. Ablation Studies

Next, we present ablation studies based on subsetting of the search space, use of STE, concatenation of input and LogicLayers, simultaneous optimization of neurons and links, application of fixed thresholds, use of Gumbel-Softmax, progressive freezing, and pruning. In Table VI, we show the average DLN accuracy and rankings based on both accuracy and the number of basic logic gate-level operations. Rank is derived based on placing the corresponding column in Tables III and IV. The DLN results are reproduced in the first column from those two tables. These original DLN results are derived under subsetting both gate and link search space to 8, using STE in all layers, and concatenating different ThresholdLayers to all middle LogicLayers. In the other columns, we list differences relative to the original setting. For freezing and pruning, we run a 32-sample hyperparameter search based on the hyperparameters of the original model. We search over only the parameters related to freezing and pruning: starting epoch for freezing, starting and ending epochs, and convergence threshold for pruning. For other cases, we perform hyperparameter search, training, and evaluation similar to the one performed for the original model.

Subset Search Space: We present results of searching over the full space, using a link subspace of size 16 for inputs of each neuron, and using both a link and gate subspace of size 4. We found that size 8 represents a local optimum.

STE: We show the results of when STE is not applied. On average, these results are worse.

TABLE VI
AVERAGE BALANCED-CLASS TEST ACCURACY PERCENTAGE OF DIFFERENT METHODS AND THEIR RANKS BASED ON THE NUMBER OF BASIC LOGIC-GATE OPERATIONS

	Orig	Subspace			STE	Concat		Phase	Fixed	Gumbel	Freeze	Prune
		w/o	16	4	w/o	w/o	same	w/o	Thresh			
BankChurn	75.8	75.7	76.4	76.4	76.9	75.3	74.1	77.1	76.5	76.1	75.2	76.7
BreastCancer	65.1	63.8	63.6	64.1	59.5	63.9	65.0	63.8	64.7	62.6	66.8	65.0
Cardiotocography	97.7	97.3	97.6	97.1	97.7	97.8	97.8	97.7	97.7	97.4	97.6	97.7
CKD	99.1	99.6	99.6	98.3	99.2	99.1	99.1	99.6	98.5	99.6	99.8	99.0
Cirrhosis	54.0	51.7	51.3	52.6	51.6	49.7	54.2	53.8	49.6	56.0	53.4	54.9
Compas	67.6	67.5	67.6	67.2	67.2	67.2	67.2	66.6	68.0	67.1	67.5	67.4
CreditFraud	94.3	93.7	93.6	93.9	94.1	94.5	94.5	93.1	93.0	94.1	95.0	94.1
Diabetes	88.0	84.5	81.1	85.5	84.7	85.1	83.5	83.2	86.8	85.6	85.0	80.5
EyeMovements	55.0	55.2	55.5	55.2	51.2	54.5	54.0	53.3	56.9	55.0	54.8	55.3
HAR	87.1	85.7	87.2	87.6	86.7	84.0	85.4	87.2	87.5	87.1	86.8	86.8
Heart	81.5	80.0	79.7	81.2	82.3	79.0	81.9	82.0	78.3	77.5	83.1	80.2
HeartFailure	68.4	69.8	69.9	71.9	68.5	71.7	65.8	71.0	66.8	76.0	67.4	67.3
HepatitisC	91.1	92.1	90.9	91.5	89.0	93.4	92.4	91.2	88.5	94.4	93.3	91.7
Letter	69.1	68.2	66.6	68.7	67.2	63.5	65.3	63.9	71.5	67.6	69.8	68.1
Liver	66.5	68.1	67.7	68.2	68.5	67.5	68.0	66.5	65.9	67.9	68.0	66.3
Mushroom	100	100	100	99.9	100	99.9	100	100	100	100	100	99.8
PDAC	59.6	59.5	56.3	57.1	58.1	59.0	59.9	58.6	54.9	58.7	57.7	58.7
SatIm	86.4	85.7	85.7	86.2	82.5	84.6	85.1	86.9	86.5	86.0	86.4	85.5
Stroke	76.2	75.4	75.3	74.3	73.9	75.6	75.6	74.7	74.5	73.1	75.0	74.7
TelcoChurn	75.9	75.2	75.7	74.9	75.1	75.5	74.7	75.0	75.9	75.1	75.4	75.7
Accuracy Rank	2.3	2.6	2.7	2.7	2.9	2.8	2.6	2.6	3.2	2.5	2.3	2.7
OPs Rank	2.3	2.1	2.2	2.5	2.2	2.0	2.1	2.5	2.9	2.2	2.3	2.1

Input Concatenation: We test not concatenating the ThresholdLayer with the LogicLayer and concatenating the same ThresholdLayer to every middle LogicLayer. We conclude that using different ThresholdLayers is better than using the same ThresholdLayer, and that itself is better than not concatenating.
Unified Phase: The results indicate that optimizing neuron functionalities and link connections in an alternating fashion is better in terms of both accuracy and model size than optimizing them together.
Fixed ThresholdLayer: We present results when the ThresholdLayers are initialized and frozen, i.e., only LogicLayers and SumLayers are trained. This not only reduces model accuracy but also results in larger models. Thus, it is better to train the ThresholdLayers.
Gumbel-Softmax: We replace Softmax activations with Gumbel-Softmaxes, using a Gumbel noise scale of 0.5. However, this does not improve the model.
Progressive Freezing: Based on the observation that layers closer to inputs perform more pattern extraction and layers closer to outputs perform more abstract pattern aggregation, we experiment with progressively freezing layer parameters to enable better gradient flow as training progresses. Results show that progressive freezing does not significantly improve model performance.
Progressive Pruning: Some neurons are not helpful in making predictions, e.g., a neuron ANDed with an always False neuron. Neurons that do not converge well may be redundant or even harmful to prediction accuracy. Hence, we test pruning out neurons that do not converge well (i.e., probabilities do not concentrate on one type of gate) in a progressive manner. However, this does not improve model accuracy. One reason may be that a larger DLN architecture search space is needed when applying pruning. Another reason may be that a non-useful neuron is already ignored in the link search phase.

F. Limitations

A shortcoming of a DLN is its training cost. To train a logic network, gradient descent must traverse layers of Softmax functions, which results in both slower backpropagation and vanishing gradients. Techniques like subsetting the search space and breaking the optimization problem into two phases help with training but do not solve the problem completely. Another limitation is that a DLN does not always perform well. For example, in Table III, on the Letter dataset, DLN underperforms MLP and some traditional models by a large margin. This may be due to the fact that rule-based approaches are not suitable for the Letter dataset since tree-based models also do not perform well.

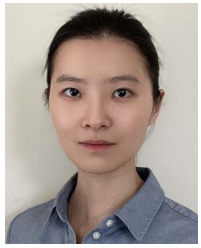
V. CONCLUSION AND FUTURE DIRECTIONS

We introduced a methodology for training DLNs whose inference processes can be explained through logic rules. The training methodology involves two iterative phases: one for determining neuron functions and another for determining neuron connections. We demonstrated that DLNs learned using this methodology are accurate, interpretable, and efficient. In the future, we plan to explore DLNs in the context of ensemble frameworks such as bagging and boosting. This would involve adding aggregation sources for class nodes. We will also explore incorporation of prior rule-based knowledge into DLNs through addition of logic paths from class nodes to the rules or representing these rules using trainable neurons.

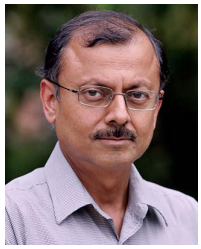
REFERENCES

[1] F. Petersen, C. Borgelt, H. Kuehne, and O. Deussen, "Deep differentiable logic gate networks," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 2006–2018.
[2] L. A. Zadeh, "Fuzzy sets as a basis for a theory of possibility," *Fuzzy Sets Syst.*, vol. 1, no. 1, pp. 3–28, 1978.

[3] K. Menger, "Statistical metrics," *Selecta Math.*, vol. 2, pp. 433–435, May 2003.
[4] B. Goertzel, M. Iklé, I. F. Goertzel, and A. Heljakka, *Probabilistic Logic Networks: A Comprehensive Framework for Uncertain Inference*. New York, NY, USA: Springer, 2008.
[5] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with Gumbel-Softmax," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–13.
[6] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," 2013, *arXiv:1308.3432*.
[7] M. T. Ribeiro, S. Singh, and C. Guestrin, "'Why should I trust you?' Explaining the predictions of any classifier," in *Proc. ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2016, pp. 1135–1144.
[8] K. Simonyan, A. Vedaldi, and A. Zisserman, "Deep inside convolutional networks: Visualising image classification models and saliency maps," 2013, *arXiv:1312.6034*.
[9] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 4768–4777.
[10] B. Zhou, A. Khosla, A. Lapedriza, A. Oliva, and A. Torralba, "Learning deep features for discriminative localization," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 2921–2929.
[11] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nature Mach. Intell.*, vol. 1, no. 5, pp. 206–215, 2019.
[12] J. Lin, C. Zhong, D. Hu, C. Rudin, and M. Seltzer, "Generalized and scalable optimal sparse decision trees," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 6150–6160.
[13] B. Ustun and C. Rudin, "Supersparse linear integer models for optimized medical scoring systems," *Mach. Learn.*, vol. 102, no. 3, pp. 349–391, 2016.
[14] R. Riegel et al., "Logical neural networks," 2020, *arXiv:2006.13155*.
[15] M. Richardson and P. Domingos, "Markov logic networks," *Mach. Learn.*, vol. 62, no. 1–2, pp. 107–136, 2006.
[16] Z. Wang, W. Zhang, N. Liu, and J. Wang, "Scalable rule-based representation learning for interpretable classification," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 30479–30491.
[17] A. Payani and F. Fekri, "Learning algorithms via neural logic networks," 2019, *arXiv:1904.01554*.
[18] S. Shi, H. Chen, M. Zhang, and Y. Zhang, "Neural logic networks," 2019, *arXiv:1910.08629*.
[19] G. Chen, "Learning symbolic expressions via Gumbel-Max equation learner networks," 2020, *arXiv:2012.06921*.
[20] H.-T. Cheng et al., "Wide & deep learning for recommender systems," in *Proc. 1st Workshop Deep Learn. Recommender Syst.*, 2016, pp. 7–10.
[21] S. Han, J. Pool, J. Tran, and W. Dally, "Learning both weights and connections for efficient neural network," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 28, 2015, pp. 1135–1143.
[22] T. Hoefler, D. Alistarh, T. Ben-Nun, N. Dryden, and A. Peste, "Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks," *J. Mach. Learn. Res.*, vol. 22, no. 1, pp. 10882–11005, 2021.
[23] M. Courbariaux, I. Hubara, D. Soudry, R. El-Yaniv, and Y. Bengio, "Binarized neural networks: Training deep neural networks with weights and activations constrained to + 1 or -1," 2016, *arXiv:1602.02830*.
[24] I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio, "Quantized neural networks: Training neural networks with low precision weights and activations," *J. Mach. Learn. Res.*, vol. 18, no. 1, pp. 6869–6898, 2017.
[25] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A survey of quantization methods for efficient neural network inference," in *Low-Power Computer Vision*, Boca Raton, FL, USA: Chapman and Hall/CRC, 2022, pp. 291–326.
[26] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015, *arXiv:1503.02531*.
[27] J. Gou, B. Yu, S. J. Maybank, and D. Tao, "Knowledge distillation: A survey," *Int. J. Comput. Vis.*, vol. 129, no. 6, pp. 1789–1819, 2021.
[28] Y. Umuroglu et al., "FINN: A framework for fast, scalable binarized neural network inference," in *Proc. ACM/SIGDA Int. Symp. FPGA*, 2017, pp. 65–74.
[29] Y. Gorishniy, I. Rubachev, and A. Babenko, "On embeddings for numerical features in tabular deep learning," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 24991–25004.
[30] A. Meurer et al., "SymPy: Symbolic computing in Python," *PeerJ Comput. Sci.*, vol. 3, Jan. 2017, Art. no. e103, doi: 10.7717/peerj-cs.103.
[31] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2019, pp. 2623–2631.
[32] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, Nov. 2011.
[33] A. Paszke et al., "Pytorch: An imperative style, high-performance deep learning library," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 8026–8037.
[34] P. Moritz et al., "Ray: A distributed framework for emerging AI applications," in *Proc. USENIX Symp. Oper. Syst. Des. Implement.*, 2018, pp. 561–577.



Chang Yue received the B.S. degree from the University of Illinois at Urbana-Champaign, in 2016, and the M.S. degree from Stanford University, in 2018, both in electrical engineering. She is currently working toward the Ph.D. degree in electrical and computer engineering with Princeton University. Her research interests include artificial intelligence and its applications to various domains.



Niraj K. Jha (Fellow, IEEE) received the B.Tech. degree in electronics and electrical communication engineering from Indian Institute of Technology, Kharagpur, India, in 1981, and the Ph.D. degree in electrical engineering from the University of Illinois at Urbana-Champaign, IL, in 1985. He is a Professor with the Department of Electrical and Computer Engineering, Princeton University. His research interests include smart healthcare and machine learning algorithms/architectures. He has served as an Associate Director for the Andlinger Center for Energy and the Environment. He is a Co-Founder of two startup companies, NeuTigers and Illumia AI, that are, respectively, commercializing edge analytics/smart healthcare and language model research from his laboratory. He has served as the Editor-in-Chief of IEEE TRANSACTIONS ON VLSI SYSTEMS and an Associate Editor of several IEEE Transactions. He has co-authored five widely used books and 480 papers, among which are 16 award-winning papers. He has also received 28 patents. He has given several keynote speeches in the areas of nanoelectronic design/test, smart healthcare, and cybersecurity. He is a fellow of ACM, and was given the Distinguished Alumnus Award by I.I.T., Kharagpur, in 2014. He has received the Princeton Graduate Mentoring Award.